AeroLink Airline Systems Platform — Cloud-Native Distributed Web Application

Student: CB012290

Name : G.H. Laksitha Gamage

Module: Cloud Computing

Agenda

1. Introduction & Problem Statement
2. Cloud Architecture Overview
3. Infrastructure as Code (Terraform)
4. Microservices Design
5. Containerisation & CI/CD Pipeline
6. Database Strategy
7. Real-Time Data Synchronisation
8. Security & Compliance
9. Fault Tolerance & Resilience
10. Performance & Scalability Testing
11. Monitoring & Observability
12. Testing Strategy
13. Live Demo — AWS Implementation
14. Thank You & Q&A

The Challenge

Legacy problems

Monolithic architecture

Single deploy unit causing long release cycles and high blast radius.

Downtime at peak

Capacity bottlenecks cause booking outages during sales and check-in surges.

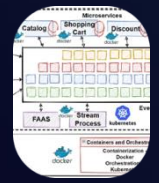
Tightly coupled components

Changes in one area often require coordinated releases across teams.

Data inconsistency

Regional differences in seat inventory and passenger state.

Proposed solution

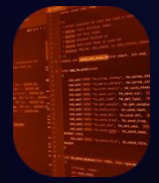


Cloud-native microservices

Small, independently deployable services with clear bounded contexts.

Event-driven architecture

Kafka topics for asynchronous coordination and eventual consistency.



Infrastructure as Code

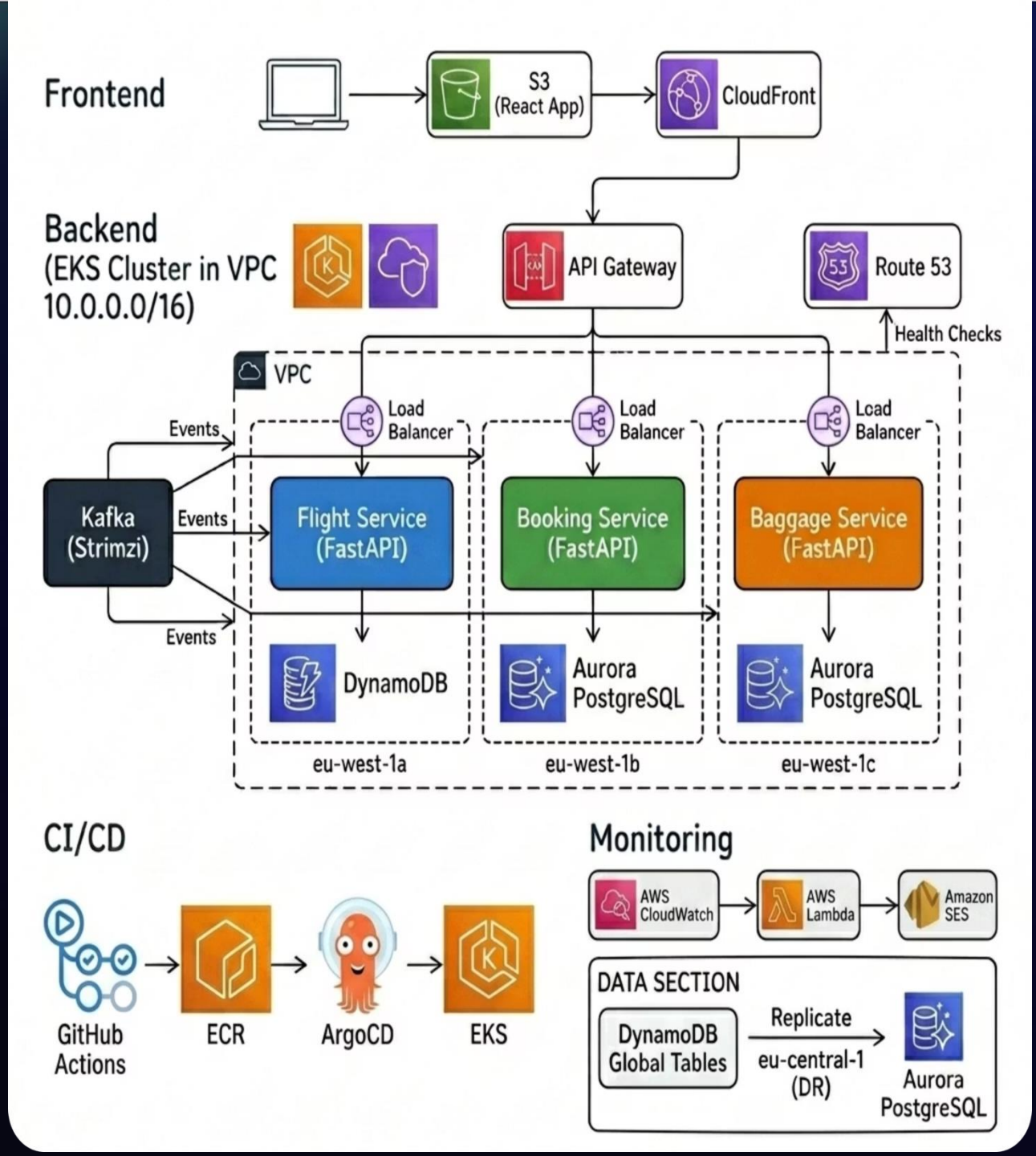
Repeatable environments via Terraform; versioned infra & GitOps practices.

Multi-region data

Global tables and managed DB clusters to reduce latency and support EU compliance.

Cloud Architecture Overview

Key components: React Frontend (S3 + CloudFront), API Gateway, EKS cluster hosting 3 microservices, Apache Kafka for events, Amazon Aurora PostgreSQL for transactional booking data, DynamoDB Global Tables for flight state, Lambda for notifications, CloudWatch for logs/metrics, GitHub Actions + ArgoCD for CI/CD.



Infrastructure as Code with Terraform

networking

VPC, subnets, routing, transit gateway

compute

EKS cluster, node groups, autoscaling

database

Aurora clusters, parameter groups, snapshots

api

API Gateway, authorizers, WAF rules

frontend

S3, CloudFront, certificate management

ecr

Private repositories, lifecycle policies

oidc

GitHub OIDC role bindings for short-lived creds

serverless

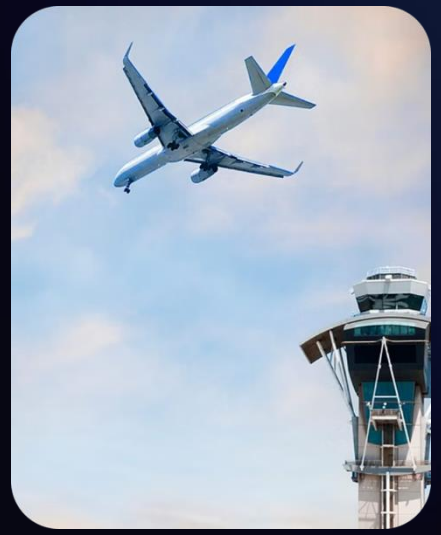
Lambda functions and event rules for notifications

monitoring

CloudWatch, alarms, log groups and dashboards

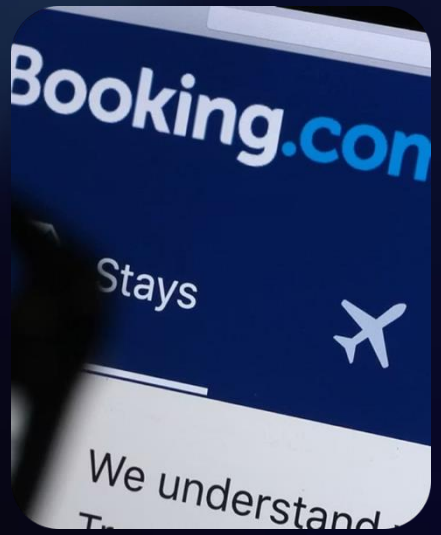
One command (terraform apply) provisions the full AWS estate — VPC, EKS, databases, API Gateway and monitoring. Infrastructure is 100% reproducible, version controlled and environment-consistent.

Three Core Microservices



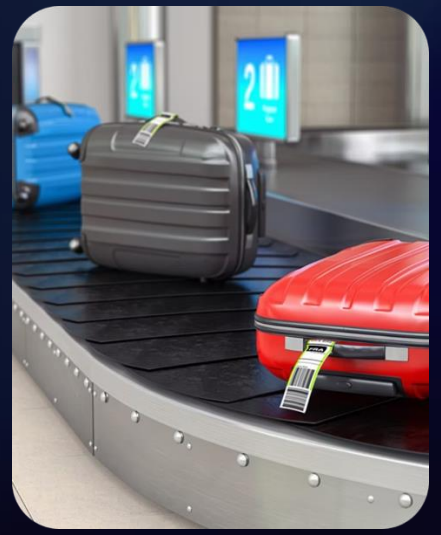
Flight Service

DynamoDB backing for schedule & seats, Kafka consumer for booking events, seat management, dynamic pricing. FastAPI + Docker, /health endpoint, k8s Deployment.



Booking Service

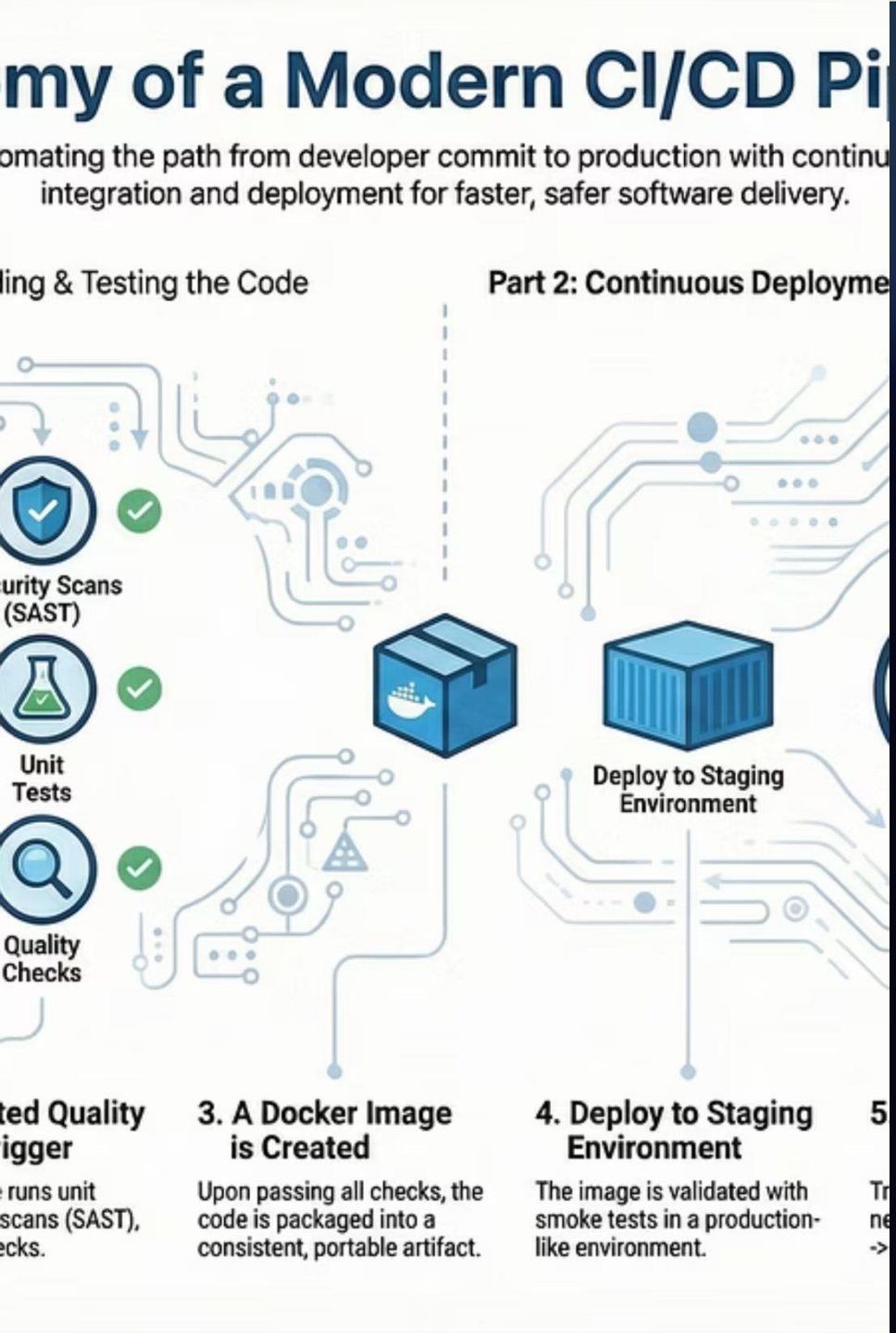
Aurora PostgreSQL for transactional bookings, Kafka producer for events, JWT auth, CRUD APIs. FastAPI + Docker, /health endpoint, k8s Deployment.



Baggage Service

DynamoDB for tracking state, real-time status updates, webhooks for external systems. FastAPI + Docker, /health endpoint, k8s Deployment.

Each service owns its data store and communicates via Kafka events. Containers are small, single-responsibility images with health probes and resource limits.



Automated Build & Deploy Pipeline

01

1. Developer push

Feature branch → pull request; linting and unit tests run in GitHub Actions.

02

2. Build & publish

Multi-stage Docker build (python:3.10-slim base), security scan, push to private ECR.

03

3. Deploy

CI updates K8s manifests; ArgoCD detects changes and reconciles to EKS.

04

4. Release

Progressive rollout with health checks and auto rollback on failure.

Implementation details: python:3.10-slim base image, multi-stage builds to minimise image size, three private ECR repositories (flight, booking, baggage).

Polyglot Persistence

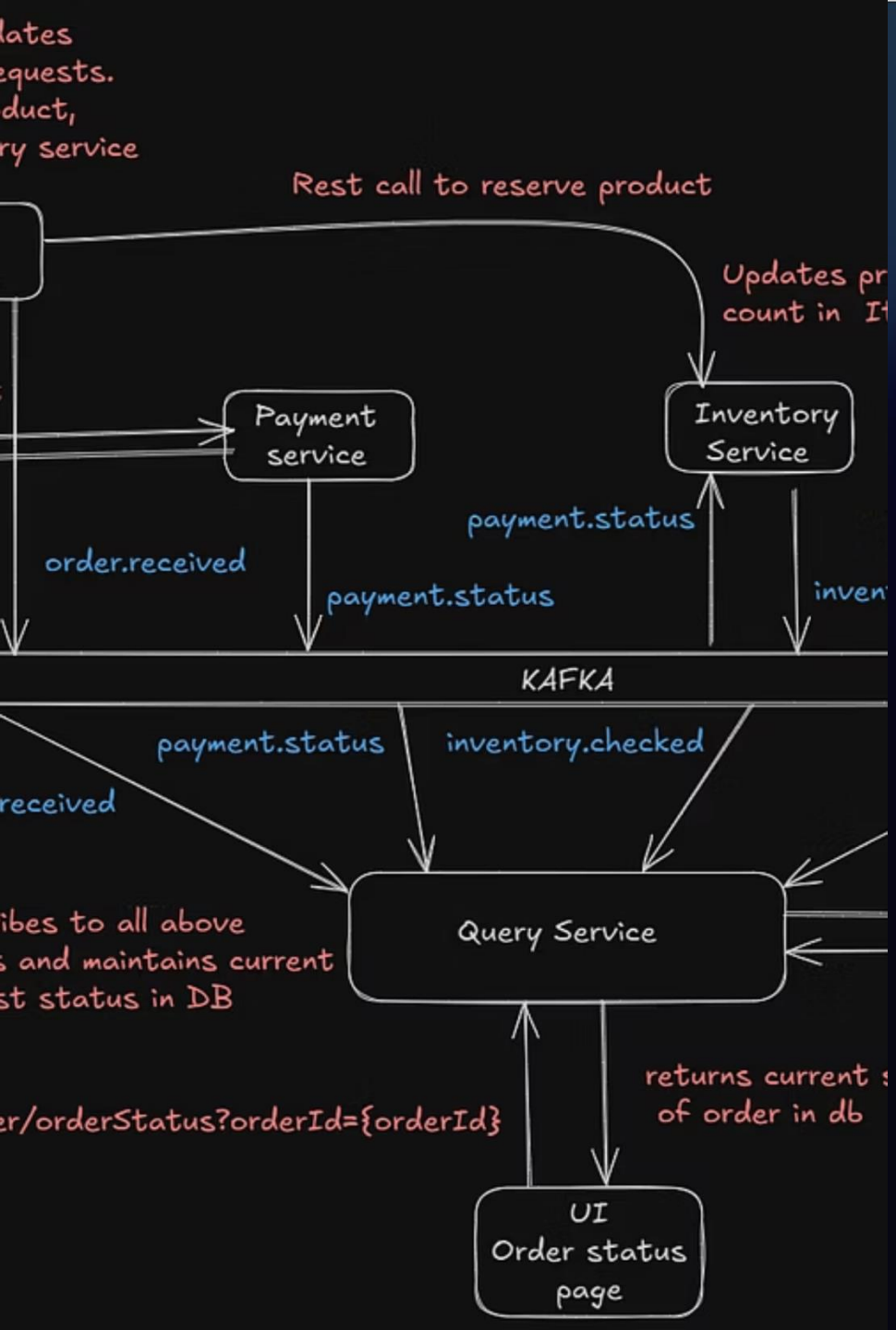
Aurora PostgreSQL

ACID transactions for booking domain. Managed RDS with read replicas, automated backups, point-in-time recovery and stored procedures for complex relational queries.

DynamoDB Global Tables

Sub-millisecond reads for flight and seat state. Replication configured between eu-west-1 and eu-central-1, with built-in conflict resolution and provisional read/write capacity via autoscaling.

Each microservice owns its own database — no shared data layer. This preserves bounded contexts and simplifies schema evolution.



Event-Driven Architecture with Kafka

1. **Booking Created**

   Booking Service produces event to booking-events topic.

2. **Flight Service consumes**

   Consumer performs conditional atomic seat decrement in DynamoDB.

3. **Saga & compensations**

   Failure triggers compensating actions or retries; fallback mode when Kafka unavailable.

Key properties: eventual consistency, Saga pattern for cross-service workflows, idempotent consumers, and DynamoDB Global Tables for multi-region convergence.

Security by Design

OAuth 2.0 + JWT

Token-based auth with 30-minute expiry; refresh tokens rotated securely. Authorizers at API Gateway and service level.

RBAC

Role separation (Admin vs Passenger). Endpoint-level access control and least privilege IAM policies.

GDPR Compliance

EU data locality (Ireland), detailed access logging, erasure endpoints and data retention controls.

PCI-DSS

Payment processing delegated to Stripe; TLS everywhere and strict network segmentation. All infra tagged Compliance = GDPR-EU via Terraform.

Security controls are enforced at multiple layers (network, infra, app). Terraform tags and automated policy checks ensure continuous compliance.

Designed for Failure

This architecture assumes components will fail and focuses on graceful degradation, rapid recovery, and automated mitigation. Key resilient mechanisms are implemented across application, infrastructure, and DNS layers to keep AeroLink available under real-world failure modes.

Circuit Breakers

Client-side and service-level circuit breakers protect downstream systems. Examples: Kafka consumer fallback modes and DynamoDB retry/backoff with alternate read paths.

Auto-Scaling

Horizontal Pod Autoscaler configured for 2–5 pods per deployment; cluster autoscaling maintains 2–5 nodes per node group for capacity smoothing and cost control.

Multi-AZ Deployment

Production runs across 3 AZs (eu-west-1a, eu-west-1b, eu-west-1c) to minimize blast radius and enable fast failover at the AZ level.

Health Checks

Readiness and liveness probes on every service ensure only healthy pods receive traffic and enable Kubernetes to restart unhealthy containers automatically.

Disaster Recovery

DynamoDB Global Tables replicate to another region (cross-region replication) for RTO/RPO goals and to permit rapid failover in the event of a full-region outage.

Recovery targets (observed / targeted): Pod crash < 30s; Node failure detection < 2min; AZ failure detection < 1min with automated rescheduling; Region failover orchestration < 5min. These SLAs are met through probes, autoscaling, and automated DNS failover.

JMeter Load & Stress Testing

Performance testing validated both steady-state and overload behavior. Results guided capacity and optimization recommendations to improve latency, error rates, and throughput under realistic user loads.

Load Test — 100 Users

Avg latency: 210 ms · Median: 164 ms · Throughput: 34.9 req/s · Error rate: 11.11% (errors concentrated in authentication flows; expected under test credentials).

Stress Test — 500 Users

Avg latency: 4,812 ms · Throughput: 96.4 req/s · Error rate: 17.11%. HPA triggered successfully, but tail latencies and auth-related failures increased.

Recommendations

- Enable connection pooling at service-to-database and service-to-cache layers
- Introduce Redis caching for read-heavy endpoints and session tokens
- Use CloudFront CDN to offload static assets and reduce origin CPU
- Right-size EC2 instances or choose burstable/compute-optimized families where appropriate

Full-Stack Observability

A layered observability strategy provides telemetry from infrastructure to code, enabling rapid detection, root-cause analysis, and SRE-driven remediation.

CloudWatch Container Insights

DaemonSet exports pod and node metrics, enabling resource visibility and anomaly detection.

Custom Dashboards

EKS failed nodes, RDS CPU, Lambda errors, and key business metrics surfaced for engineers and ops.

Centralized Logging

Structured Python application logs plus API Gateway access logs shipped to CloudWatch/Elasticsearch for search and retention.

Distributed Tracing

OpenTelemetry annotations enable end-to-end traces across services to measure latency contributions and dependency maps.

Health Endpoints

Each service exposes /health for liveness/readiness; used by Kubernetes and Route 53 health checks for traffic routing decisions.

Route 53 Health Checks

DNS-level monitoring with failover policies to redirect traffic during regional incidents.

Together these layers reduce mean time to detection and repair (MTTD/MTTR) and provide the telemetry needed to validate SLOs.

Comprehensive Testing Pyramid

The testing strategy follows a pyramid approach: many fast unit tests at the base, a smaller set of integration tests, and focused API tests for end-to-end contract verification. This mix balances speed with confidence for CI pipelines.

Unit Tests

15 unit tests total (6 Booking, 5 Flight, 4 Baggage) using pytest + FastAPI TestClient. Observed 100% pass rate consistently across every run in CI.

Integration Tests

End-to-end flows exercised across EKS-deployed services, API Gateway validation, and RBAC enforcement tests to ensure behavior under real deployment conditions.

API Tests & Documentation

Postman collection with automated assertions for critical endpoints and Swagger UI available for interactive contract testing and exploratory QA.

Highlight: All 15 unit tests passed consistently on every execution — a key confidence signal for CI/CD promotions.

LIVE DEMO

AWS IMPLEMENTATION

Live Demo — AWS Implementation

Proof of Deployment from the AWS Console

Gallery items correspond to live resources: EKS cluster, ECR images, DynamoDB Global Tables, CloudWatch dashboards, API Gateway routes, ArgoCD sync status, GitHub Actions runs, S3-hosted frontend, and active ELB. Use the live console to demonstrate logs, pod details, and scaling events in real time.

Thank You

Questions & Answers

Repository

https://github.com/HirushaGamage10/aerolink

Student ID

G.H. Laksitha Gamage

CB012290